

Dover GaiN SE User Manuals

Codex - User Manual

Version: April 2026

Overview

Codex provides a flexible development environment that can be installed through either the VS Code extension or the Codex desktop application, enabling users to begin working within a familiar interface. A standard setup requires secure configuration of the GAIN_API_KEY along with creation of the primary configuration files, such as config.toml and gain-models.json, which define authentication and model behavior. After installation, users can select an appropriate GaiN model profile—Standard, Coder, Pro, or Advanced—according to the complexity and nature of their tasks. With the environment configured, users can initialize projects, understand Codex file storage conventions, and begin using core capabilities including file operations, memory, plugins and MCP integrations, skills, browser and computer interaction, image generation, and workflow automation. The platform also supports the creation and reuse of custom skills to streamline repeated tasks and improve consistency across workflows. To ensure efficient MCP usage, it is recommended to narrow request scope, cache reusable outputs, and adopt search-first patterns before issuing broader calls. This guide also addresses common setup and configuration issues to help users troubleshoot effectively and maintain a stable, productive Codex environment.

Prerequisites

Make sure you have:

- A Windows or macOS computer.
- Access to the Codex VS Code extension or Codex desktop app.
- Your organization-provided GaiN API key.
- Permission to create files in your user profile folder.

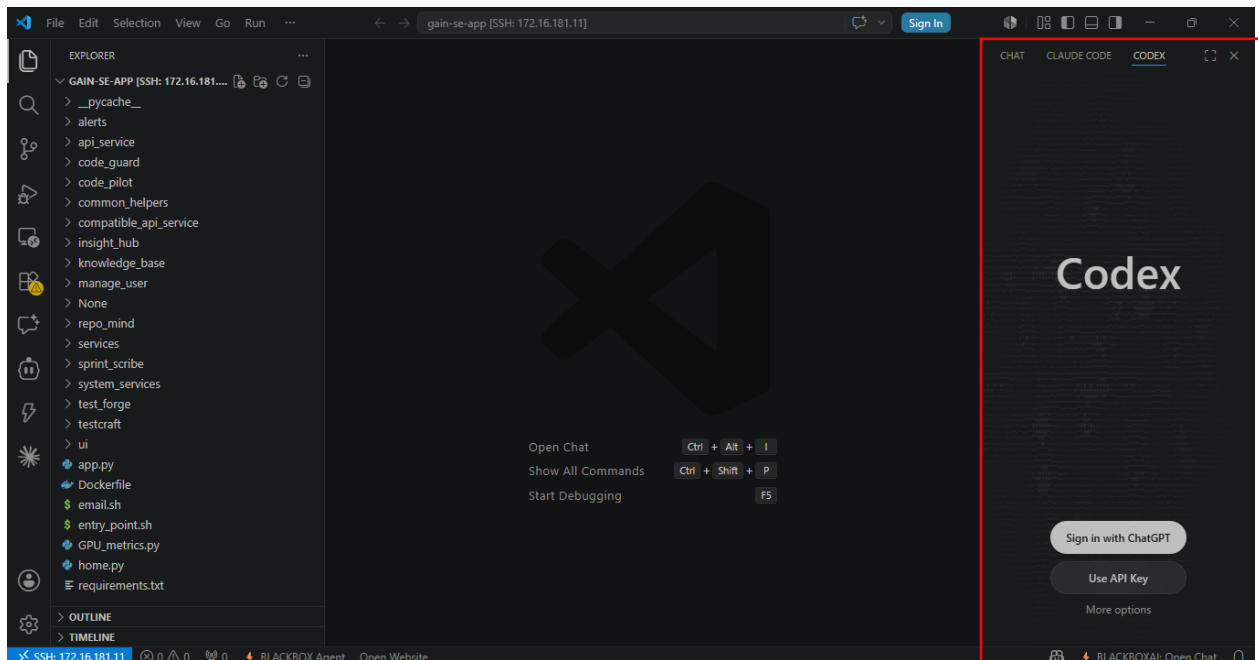
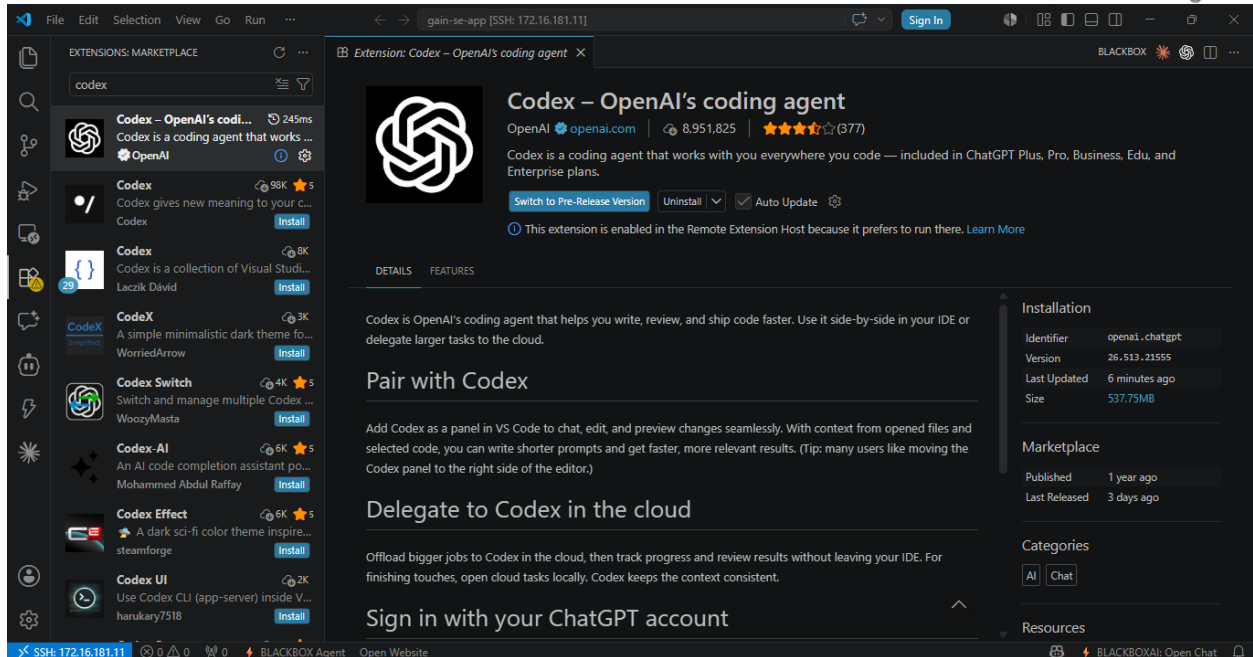
Steps for the Setup of Codex:

1. Install Codex

Install one of the following:

- **Codex VS Code extension**, if you want Codex inside Visual Studio Code.
- **Codex desktop app**, if you want a standalone app experience.

After installing, close all open windows where Codex is available. This is important because environment variable changes often require a fresh app or terminal session before they are detected.



4. Set your Gain API key

Codex will read your key from an environment variable named `GAIN_API_KEY`.

Windows: set the key permanently

Open PowerShell and run:

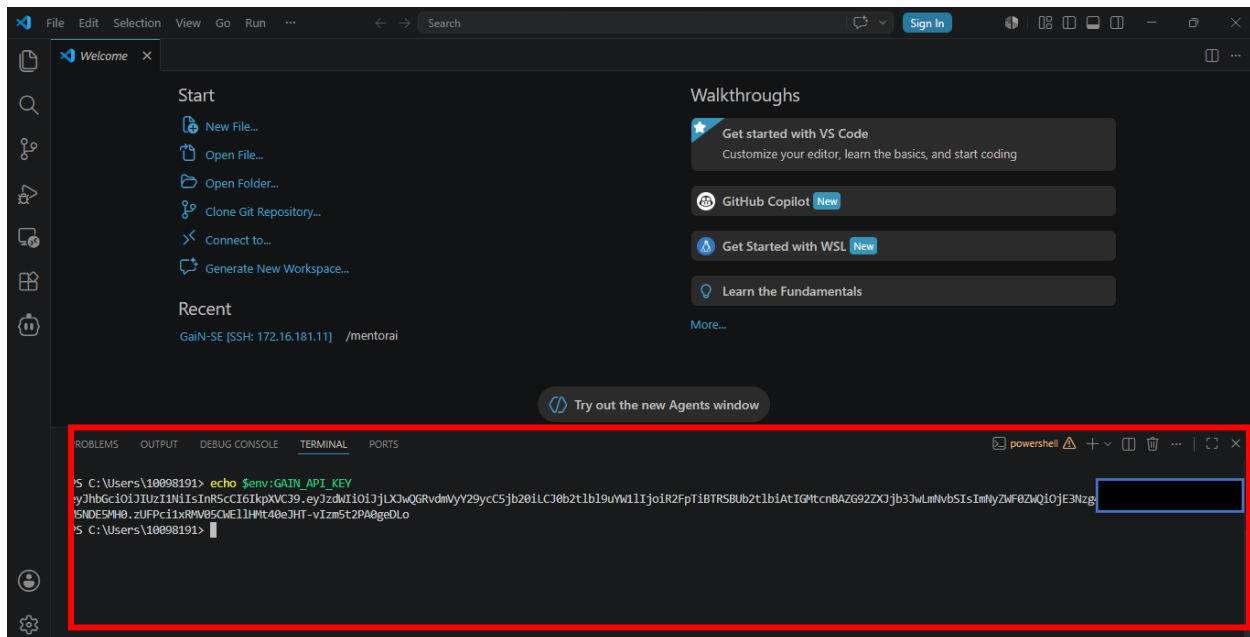
```
setx GAIN_API_KEY "PASTE_TOKEN_HERE"
```

Then close and reopen PowerShell, VS Code, and/or the Codex app.

To verify the variable exists in a new PowerShell window:

```
echo $env:GAIN_API_KEY
```

You should see a value. Do not share that value.



macOS: set the key for the current terminal session

```
export GAIN_API_KEY="PASTE_TOKEN_HERE"
```

macOS: set the key permanently

Open your shell profile:

```
nano ~/.zshrc
```

Add this line: `export GAIN_API_KEY="PASTE_TOKEN_HERE"`

Save the file, then reload it:

```
source ~/.zshrc
```

Restart Codex after changing this value.

5. Create your .codex folder

Codex needs configuration files in your user profile folder.

Windows path

C:\Users\Employee_ID\.codex\

Replace Employee_ID with your actual Windows user folder name.

Example PowerShell command:

```
New-Item -ItemType Directory -Force "$env:USERPROFILE\.codex"
```

macOS path

~/codex/

Example terminal command:

```
mkdir -p ~/.codex
```

6. Create config.toml

Create this file:

Windows

C:\Users\Employee_ID\.codex\config.toml

macOS

~/codex/config.toml

Windows config.toml example

Replace Employee_ID with your actual user folder name. Make sure there are no accidental spaces in the path.

```
model_catalog_json = "C:\\Users\\EmployeeID\\.codex\\gain-models.json"

model = "Pro"
model_reasoning_effort = "medium"
oss_provider = "lmstudio"
```

```
model_provider = "GaiN"

[profiles.Standard]
model_provider = "GaiN"
model = "Standard"

[profiles.Coder]
model_provider = "GaiN"
model = "Coder"

[profiles.Pro]
model_provider = "GaiN"
model = "Pro"

[model_providers.GaiN]
name = "GaiN"
base_url = "https://gains-api.dovercorp.com/compatible/v1/"
env_key = "GAIN_API_KEY"

[otel]
exporter = "none"
log_user_prompt = false
analytics = "none"

[mcp_servers.atlassian]
bearer_token_env_var =
"ATATT3xFfGF0oT17Hzb3gllIG5lgYPputhBfhfL7ICQTxObwkedYp7W05lOk6iRXZ6vX
QujGdSO3ldGNNFUvHur_pCPRxhDyYXwah0fxldkPW3YkWJ6IPuYVRDQdoO3SQMNI
6ROUOF-9FNRrteoqKnzCEhgZ0Yfma5OPS80aSmh4Y9q-FMDd0=5268A35D"
enabled = true
url = "https://dovercorp.atlassian.net"

[mcp_servers.figma]
bearer_token_env_var = "figd_0TN7KrrlWmJifV2rFzwhG-fbi8LFgxMqA6TIlx1Z"
enabled = true
url = "https://mcp.figma.com/mcp"

[mcp_servers.jira]
```

```
bearer_token_env_var =  
"ATATT3xFfGF0oT17Hzb3gllIG5lgYPputhBfihfL7ICQTxObwkedYp7W05lOk6iRXZ6vX  
QujGdSO3ldGNNFUvHur_pCPRxhDyYXwah0fxldkPW3YkWJ6IPuYVRDQdoO3SQMNI  
6ROUOF-9FNRrteoqKnzCEhgZ0Yfma5OPS80aSmh4Y9q-FMDd0=5268A35D"  
enabled = true  
url = "https://dovercorp.atlassian.net"
```

macOS config.toml example

```
model_catalog_json = "/Users/Employee_ID/.codex/config.toml"  
  
model = "Pro"  
model_reasoning_effort = "medium"  
oss_provider = "lmstudio"  
model_provider = "GaiN"  
  
[profiles.Standard]  
model_provider = "GaiN"  
model = "Standard"  
  
[profiles.Coder]  
model_provider = "GaiN"  
model = "Coder"  
  
[profiles.Pro]  
model_provider = "GaiN"  
model = "Pro"  
  
[model_providers.GaiN]  
name = "GaiN"  
base_url = "https://gains-api.dovercorp.com/compatible/v1/"  
env_key = "GAIN_API_KEY"  
  
[otel]  
exporter = "none"  
log_user_prompt = false  
analytics = "none"
```

Beginner tip: Use straight quotes (") instead of curly quotes (for example, smart opening/closing quotation marks). Curly quotes can break config files.

7. Create gain-models.json

Create this file:

Windows

C:\Users\Employee_ID\.codex\gain-models.json

macOS

~/.codex/gain-models.json

Paste the following JSON into the file:

```
{
  "models": [
    {
      "slug": "Standard",
      "display_name": "Standard",
      "description": "GaiN Standard model",
      "base_instructions": "You are Codex, a coding agent.",
      "default_reasoning_level": "medium",
      "supports_reasoning_summaries": true,
      "default_reasoning_summary": "auto",
      "support_verbosity": true,
      "default_verbosity": null,
      "apply_patch_tool_type": "freeform",
      "web_search_tool_type": "text",
      "truncation_policy": {
        "mode": "tokens",
        "limit": 10000
      },
      "supports_parallel_tool_calls": true,
      "supports_image_detail_original": true,
      "context_window": 256000,
    }
  ]
}
```

```
"max_context_window": 256000,
"effective_context_window_percent": 95,
"experimental_supported_tools": [],
"input_modalities": [
  "text",
  "image"
],
"supports_search_tool": true,
"supported_reasoning_levels": [
  {
    "effort": "low",
    "description": "Fast responses with lighter reasoning"
  },
  {
    "effort": "medium",
    "description": "Balances speed and reasoning depth"
  },
  {
    "effort": "high",
    "description": "Greater reasoning depth"
  },
  {
    "effort": "xhigh",
    "description": "Extra high reasoning depth"
  }
],
"shell_type": "shell_command",
"visibility": "list",
"supported_in_api": true,
"priority": 0
},
{
  "slug": "Coder",
  "display_name": "Coder",
  "description": "GaiN Coder model",
  "base_instructions": "You are Codex, a coding agent.",
  "default_reasoning_level": "medium",
  "supports_reasoning_summaries": true,
  "default_reasoning_summary": "auto",
```



```
"support_verbosity": true,
"default_verbosity": null,
"apply_patch_tool_type": "freeform",
"web_search_tool_type": "text",
"truncation_policy": {
  "mode": "tokens",
  "limit": 10000
},
"supports_parallel_tool_calls": true,
"supports_image_detail_original": true,
"context_window": 256000,
"max_context_window": 256000,
"effective_context_window_percent": 95,
"experimental_supported_tools": [],
"input_modalities": [
  "text",
  "image"
],
"supports_search_tool": true,
"supported_reasoning_levels": [
  {
    "effort": "low",
    "description": "Fast responses with lighter reasoning"
  },
  {
    "effort": "medium",
    "description": "Balances speed and reasoning depth"
  },
  {
    "effort": "high",
    "description": "Greater reasoning depth"
  },
  {
    "effort": "xhigh",
    "description": "Extra high reasoning depth"
  }
],
"shell_type": "shell_command",
"visibility": "list",
```

```
"supported_in_api": true,
"priority": 1
},
{
  "slug": "Pro",
  "display_name": "Pro",
  "description": "GaiN Pro model",
  "base_instructions": "You are Codex, a coding agent.",
  "default_reasoning_level": "medium",
  "supports_reasoning_summaries": true,
  "default_reasoning_summary": "auto",
  "support_verbosity": true,
  "default_verbosity": null,
  "apply_patch_tool_type": "freeform",
  "web_search_tool_type": "text",
  "truncation_policy": {
    "mode": "tokens",
    "limit": 10000
  },
  "supports_parallel_tool_calls": true,
  "supports_image_detail_original": true,
  "context_window": 256000,
  "max_context_window": 256000,
  "effective_context_window_percent": 95,
  "experimental_supported_tools": [],
  "input_modalities": [
    "text",
    "image"
  ],
  "supports_search_tool": true,
  "supported_reasoning_levels": [
    {
      "effort": "low",
      "description": "Fast responses with lighter reasoning"
    },
    {
      "effort": "medium",
      "description": "Balances speed and reasoning depth"
    }
  ],
}
```

```
{
  "effort": "high",
  "description": "Greater reasoning depth"
},
{
  "effort": "xhigh",
  "description": "Extra high reasoning depth"
}
],
"shell_type": "shell_command",
"visibility": "list",
"supported_in_api": true,
"priority": 2
},
{
  "slug": "Advanced",
  "display_name": "Advanced",
  "description": "GaiN Advanced model",
  "base_instructions": "You are Codex, a coding agent.",
  "default_reasoning_level": "medium",
  "supports_reasoning_summaries": true,
  "default_reasoning_summary": "auto",
  "support_verbosity": true,
  "default_verbosity": null,
  "apply_patch_tool_type": "freeform",
  "web_search_tool_type": "text",
  "truncation_policy": {
    "mode": "tokens",
    "limit": 10000
  },
  "supports_parallel_tool_calls": true,
  "supports_image_detail_original": true,
  "context_window": 256000,
  "max_context_window": 256000,
  "effective_context_window_percent": 95,
  "experimental_supported_tools": [],
  "input_modalities": [
    "text",
    "image"
```

```
],  
  "supports_search_tool": true,  
  "supported_reasoning_levels": [  
    {  
      "effort": "low",  
      "description": "Fast responses with lighter reasoning"  
    },  
    {  
      "effort": "medium",  
      "description": "Balances speed and reasoning depth"  
    },  
    {  
      "effort": "high",  
      "description": "Greater reasoning depth"  
    },  
    {  
      "effort": "xhigh",  
      "description": "Extra high reasoning depth"  
    }  
  ],  
  "shell_type": "shell_command",  
  "visibility": "list",  
  "supported_in_api": true,  
  "priority": 3  
}  
]  
}
```

After saving both files, restart Codex.

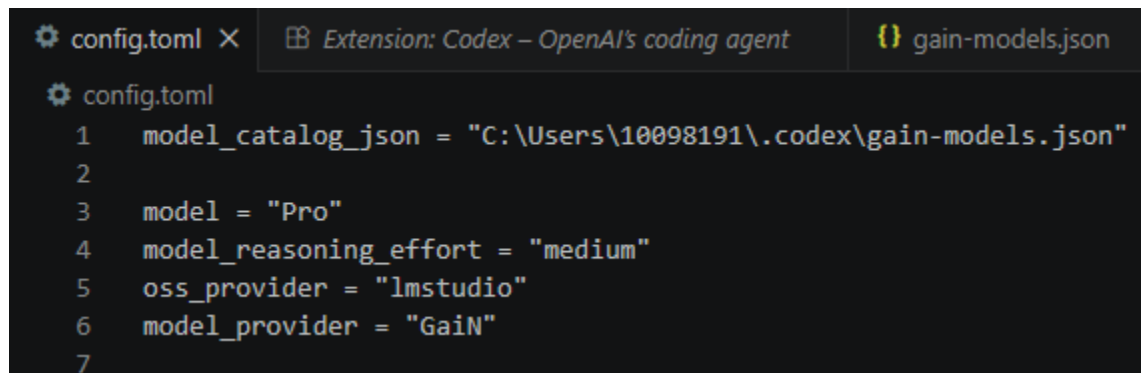
8. Choose your model profile

The setup document defines these profiles:

Profile	Guidance
Standard	Good for everyday questions and lighter tasks.
Coder	Best starting point for coding tasks.
Pro	Strong default for more complex work; this tutorial configures it as the default.
Advanced	Use when you need deeper reasoning or more capable analysis.

The default in config.toml is:

```
model = "Pro"
model_reasoning_effort = "medium"
```

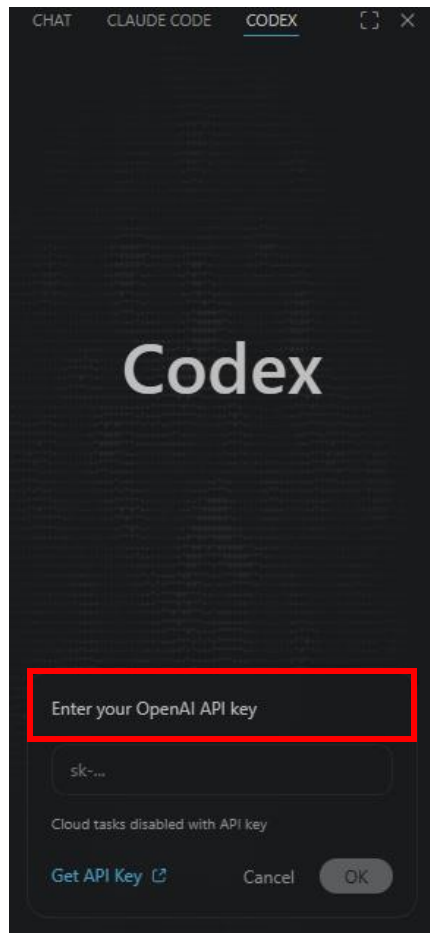


```
config.toml
1  model_catalog_json = "C:\Users\10098191\.codex\gain-models.json"
2
3  model = "Pro"
4  model_reasoning_effort = "medium"
5  oss_provider = "lmstudio"
6  model_provider = "Gain"
7
```

That is a good beginner default because it balances capability and speed.

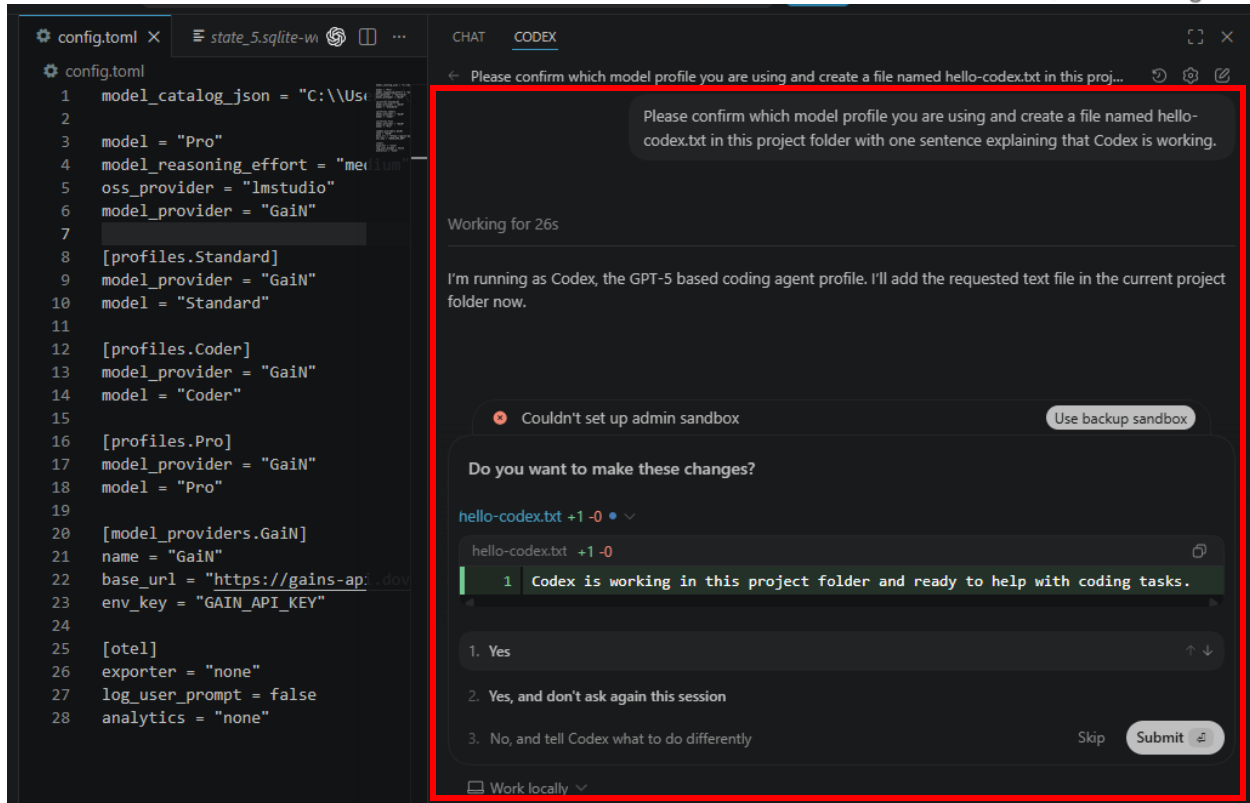
9. Verify your setup

SIGN IN INTO THE CODEX USING THE API TOKEN



Open Codex and start a new chat. Try this prompt:

Please confirm which model profile you are using and create a file named hello-codex.txt in this project folder with one sentence explaining that Codex is working.



Then check:

1. Did Codex answer without an API-key error?
2. Did it create the file in your project folder?
3. Can you open the file locally?

If yes, your basic setup is working.

Atlassian, Jira and Figma MCP Setup for Codex

1. Atlassian / Jira MCP Setup in Codex

Configuration Values

Name: atlassian

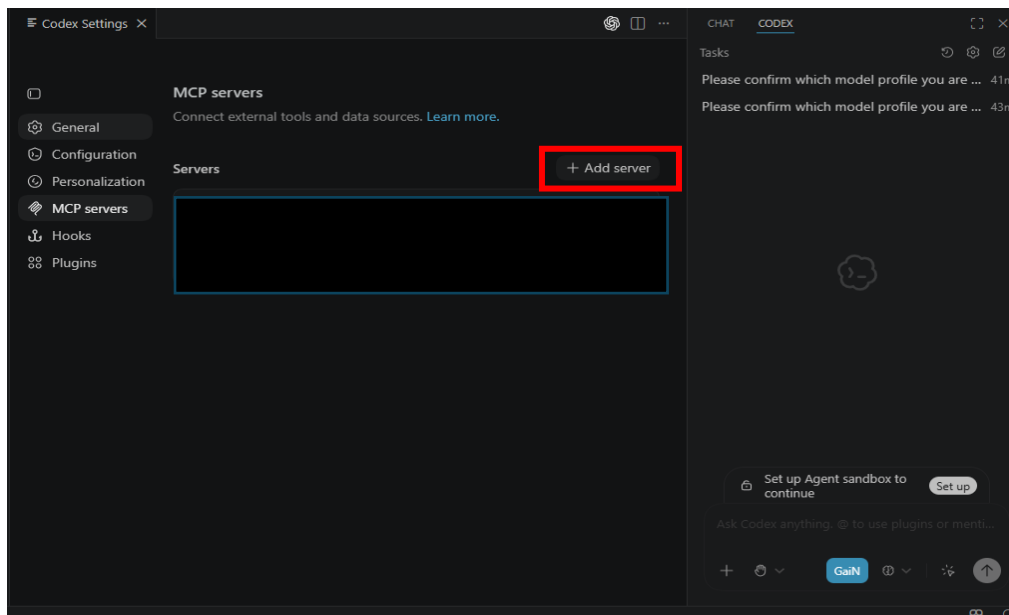
Type: Streamable HTTP

URL: <https://mcp.atlassian.com/v1/mcp/authv2>

API KEY: YOUR_CONFLUENCE_API_TOKEN

Steps

1. Open Codex.
2. Go to Settings.
3. Open MCP servers.
4. Click Add server.



5. Set the server type to Streamable HTTP.
6. In the Name field, enter: atlassian
7. In the URL field, enter:
<https://mcp.atlassian.com/v1/mcp/authv2>
8. Setup your API KEY as an environment variable using the following command:
`setx ATlassian_TOKEN "YOUR_ATlassian_API_KEY"`
9. Enter the environment variable name in Bearer Token env var box
10. Click Save.

sample.py Codex Settings X gain-models.json

← Back

Update Atlassian MCP Uninstall

If you would like to switch MCP server type, please uninstall first.

URL

`https://mcp.atlassian.com/v1/mcp/authv2`

Bearer token env var

[Redacted]

Headers

Key	Value	
		🗑️
+ Add header		

Headers from environment variables

Key	Value	
		🗑️
+ Add variable		

Save

11. Approve the requested permissions and restart codex.

12. After setup is complete, test with a Jira-related prompt in Codex.

Recommended Test Prompts

- List my Jira issues
- Show open Jira tickets assigned to me
- Search Jira issues in my project

2. Jira Setup in Codex

There is no separate Jira-only MCP endpoint required for this configuration.

Jira access is provided through the Atlassian MCP server.

Configuration Values

Name: jira

Type: Streamable HTTP

URL: <https://mcp.atlassian.com/v1/mcp/authv2>

API KEY: YOUR_JIRA_API_TOKEN

3. Figma MCP Setup in Codex

Configuration Values

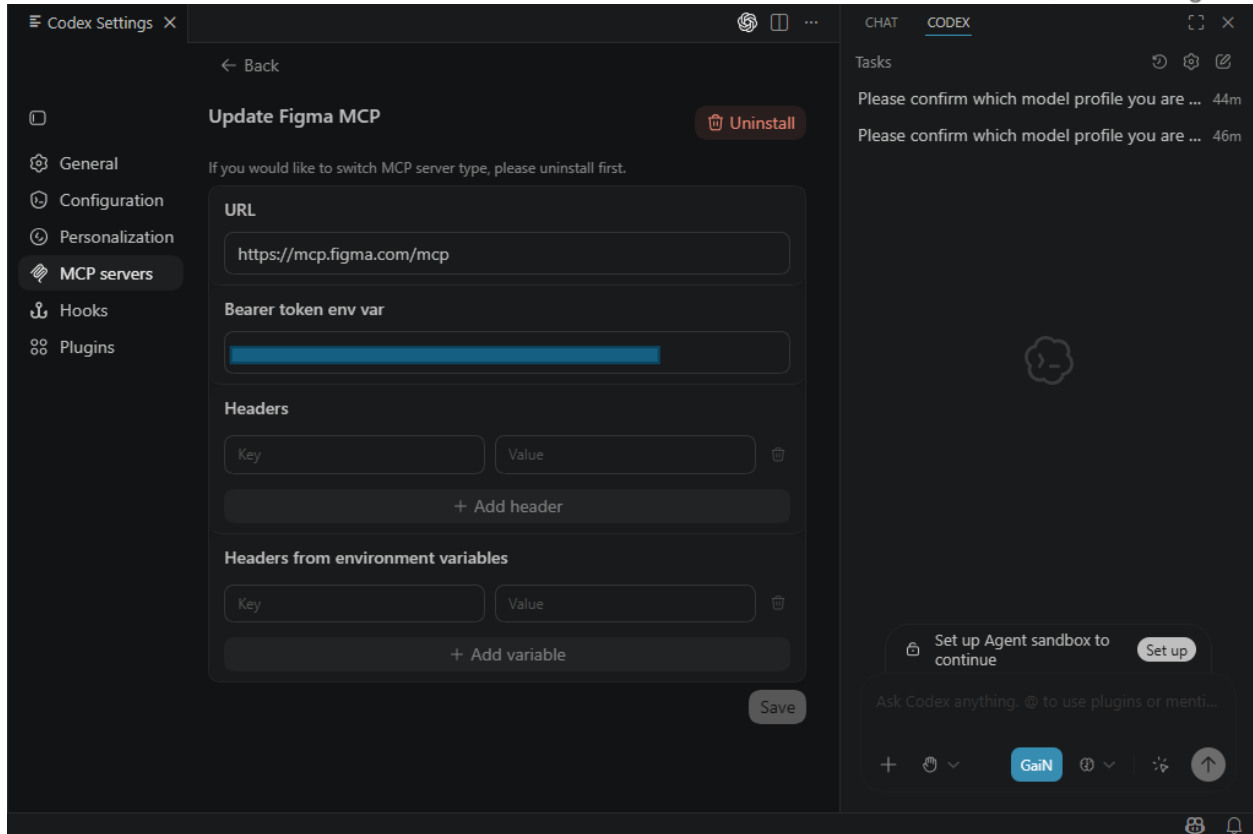
Name: figma

Type: Streamable HTTP

URL: <https://mcp.figma.com/mcp>

Steps

1. Open Codex.
2. Go to Settings.
3. Open MCP servers.
4. Click Add server.
5. Set the server type to Streamable HTTP.
6. In the Name field, enter:
figma
7. In the URL field, enter:
<https://mcp.figma.com/mcp>
8. Setup your API KEY as an environment variable using the following command:
setx FIGMA_TOKEN "YOUR_FIGMA_API_KEY"
9. Enter the environment variable name in Bearer Token env var box
10. Click Save.



11. Complete the Figma sign-in or authorization flow if prompted and restart codex.
12. After setup is complete, test with a Figma-related prompt in Codex.

Recommended Test Prompts

- Read this Figma file and summarize the components
- Show variables used in this Figma design
- Summarize the layout structure from the Figma file

5. Example config.toml Entries

Use the following entries if you want to configure the same servers directly in Codex config.

```
[mcp_servers.atlassian]

url = "https://mcp.atlassian.com/v1/mcp/authv2"

[mcp_servers.figma]
```

```
url = "https://mcp.figma.com/mcp"
```

Support

For installation help, troubleshooting, or feedback, please contact your Gain SE support team. Gains-support@dovercorp.com